Object Oriented Programming with C#

Lâm Hoài Bảo

lhbao@ctu.edu.vn

Content

- **Classes**
- Inheritance
- Interface
- Summary

Class and Object

Class

- Common attributes and activities of instances are abstracted as class.

- **EX**: Clas Person:
  - Name
  - Height
  - Hair color.
  - Talk
  - Write
  - ….
- Attributes, activities: member

Object

- An object is a particular instance of a class.
- **EX**: An object Person:

  - Name: Van Cuong
  - Age: 29
  - Weight: 60 kg

  Activies:

  - Go
  - Talk
  - Think

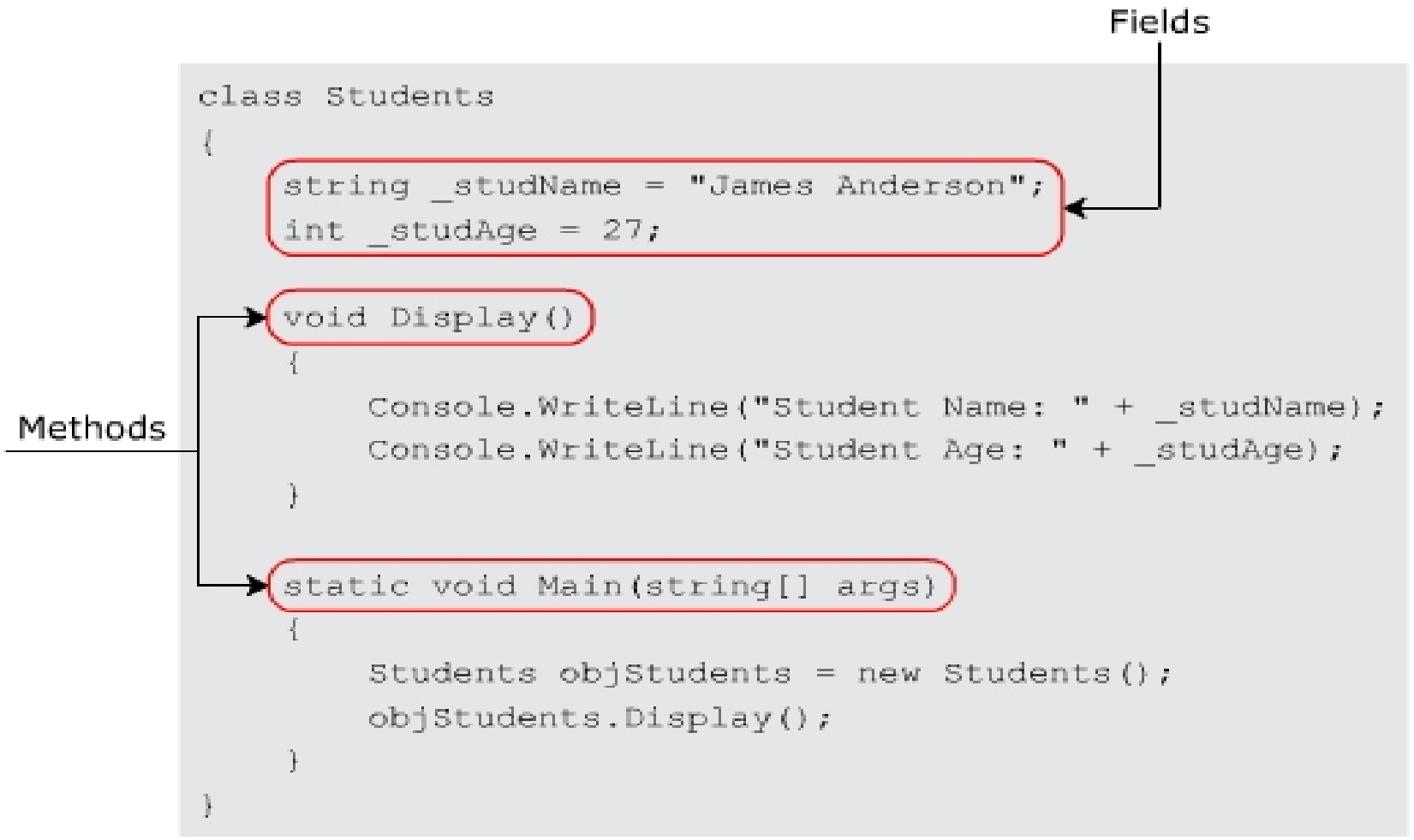
Declare a new class

- Keyword class:

```
class <Class name>{
        <Class members>
}
```

- <Class members>: data, methods

Declare a new class



Create object

- **Declare and initialize** a variable of a class.
- EX: Object p of Students class:

```
Students p;
p = new Students();
//or
Students p = new Students ();
```

Method

Method 1

- Object activities are implemented as methods (functions)
- Declaration:

  [<Access modifier>] <Return type> <Method name>
  ([<Parameter declaration>])
  {
  <Declaration, Statements>
  }
- Parameter declaration ~ Variable declaration.
- Method call:

  <Method name> (<Real parameters>)

Method [2]

```
class Book
{
      string _bookName;
      public string Print()
      {
            return _bookName;
      }

      public void Input(string _bName)
      {
            _bookName=_bName;
      }

      static void Main()
      {
            Book b = new Book();
            b.Input("Inside C#");
            Console.WriteLine(b.Print());
            Console.ReadLine();
      }
}
```

Access modifier

- private
  - Can only be accessed within the class in which it is defined
- protected
  - Can only be accessed within the class and its subclassed
- public
  - Can be accessed from anywhere in the program

The keyword static

- Used in field and method
- Refer using class name
- Static data field: a single copy of that field is created and shared among all instances of that class

```
private static char TAB = '\t';
```

- Static method: belongs to a class (similar to static field)

```
public static void Welcome() {...}
```

static method

```
class Calculate
{
   public static void Addition(int val1, int val2)
   {
      Console.WriteLine(val1 + val2);
   }
   public void Multiply(int val1, int val2)
   {
      Console.WriteLine(val1 * val2);
   }
}
class StaticMethods
{
   static void Main(string [] args)
   {
      Calculate.Addition(10, 50);
      Calculate objCal = new Calculate();
      objCal.Multiply(10, 20);
   }
}
```

Parameter passing

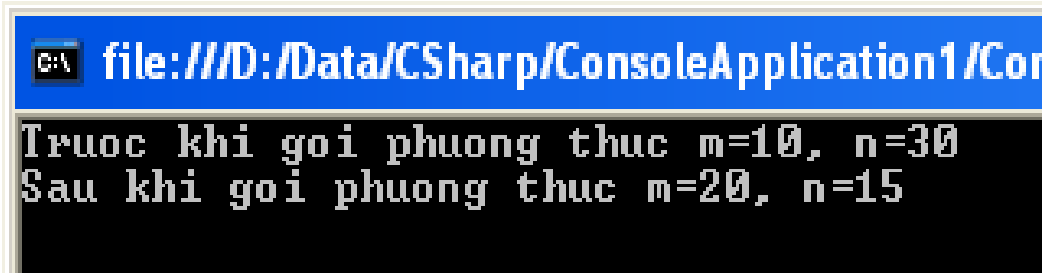
- **Pass by reference: ref** keyword is used in the method definition and calls
- Before calling the method, the real parameter must be initialized

Pass-by-referenced

```
class RefParameters
{
    static void Calculate(ref int a, ref int b)
    {
        a=2*a;
        b=b/2;
    }

    static void Main()
    {
        int m=10;
        int n=30;
        Console.WriteLine("Truoc khi goi phuong thuc m={0}, n={1}",m,n);
        Calculate(ref m, ref n);
        Console.WriteLine("Sau khi goi phuong thuc m={0}, n={1}",m,n);

        Console.ReadLine();
    }
}
```



Output parameter

- Kinda like pass-by-referenced
- Keyword **out** is used in function definition and call
- The real parameter may not be initialized before passing

Output parameters

```
class OutParameters
{
      static void Depreciation(out int val)
      {
            val = 20000;
            int dep = val * 5/100;
            int amt = val - dep;
            Console.WriteLine("Depreciation Amount: " + dep);
            Console.WriteLine("Reduced value after depreciation: "
            + amt);
      }

      static void Main(string[] args)
      {
           int value;
           Depreciation(out value);
      }
}
```

Method Overloading

Method Overloading

- Methods of a class can have the same name but different signatures
  - Signatures: Parameter list
- Can be done by changing
  - The number of parameters
  - The data type of parameters
  - The order of parameters of method

Method Overloading

```
class MethodOverloading
{
    static int Square(int num)
    {
        return num*num;
    }

    static double Square(double num)
    {
        return num*num;
    }

    static void Main()
    {
        Console.WriteLine("Binh phuong so nguyen {0}",Square(5));
        Console.WriteLine("Binh phuong so thuc {0}",Square(5.5));
        Console.ReadLine();
    }
}
```

The keyword this

- The keyword **this** refers to the current instance of the class.
- Common uses
  - Qualify members hidden by similar names
  - Pass an object to other methods
  - Declare indexers

The keyword this

```
class ThisKeyword
{
    double length, width;

    public double Area(double length, double width)
    {
        this.length = length;
        this.width = width;
        return this.length*this.width;
    }

    static void Main()
    {
        ThisKeyword obj = new ThisKeyword();

        Console.WriteLine("Dien tich la {0}",obj.Area(3.0,2.5));
        Console.ReadLine();
    }
}
```

Constructor & Destructor

Constructor

- This method is called to create an instance of a class
- In C#, the constructor has the same name with the class
- It can be overloadable.

Constructor

```
class Employee
{
    public string emp_Name;
    public int emp_Age;

    public Employee(string Name, int Age)
    {
        emp_Name=Name;
        emp_Age=Age;
    }
}

class Constructor
{
    static void Main()
    {
        Employee emp = new Employee("Roger Federer",29);
        Console.WriteLine("Name: " + emp.emp_Name + " - Age: " + emp.emp_Age);
        Console.ReadLine();
    }
}
```

```
class Rectangle
{
    double length, width;

    public Rectangle()
    {
        length=10.0;
        width=5.0;
    }

    public Rectangle(double length, double width)
    {
        this.length=length;
        this.width=width;
    }

    public double Area()
    {
        return this.length*this.width;
    }

    static void Main()
    {
        Rectangle rec1 = new Rectangle();
        Console.WriteLine("Dien tich hinh chu nhat la {0}",rec1.Area());
        Rectangle rec2 = new Rectangle(5.0,2.5);
        Console.WriteLine("Dien tich hinh chu nhat la {0}",rec2.Area());
        Console.ReadLine();
    }
}
```

Destructor

- Is called to release an object
- Naming: ~<Class name>

Destructor

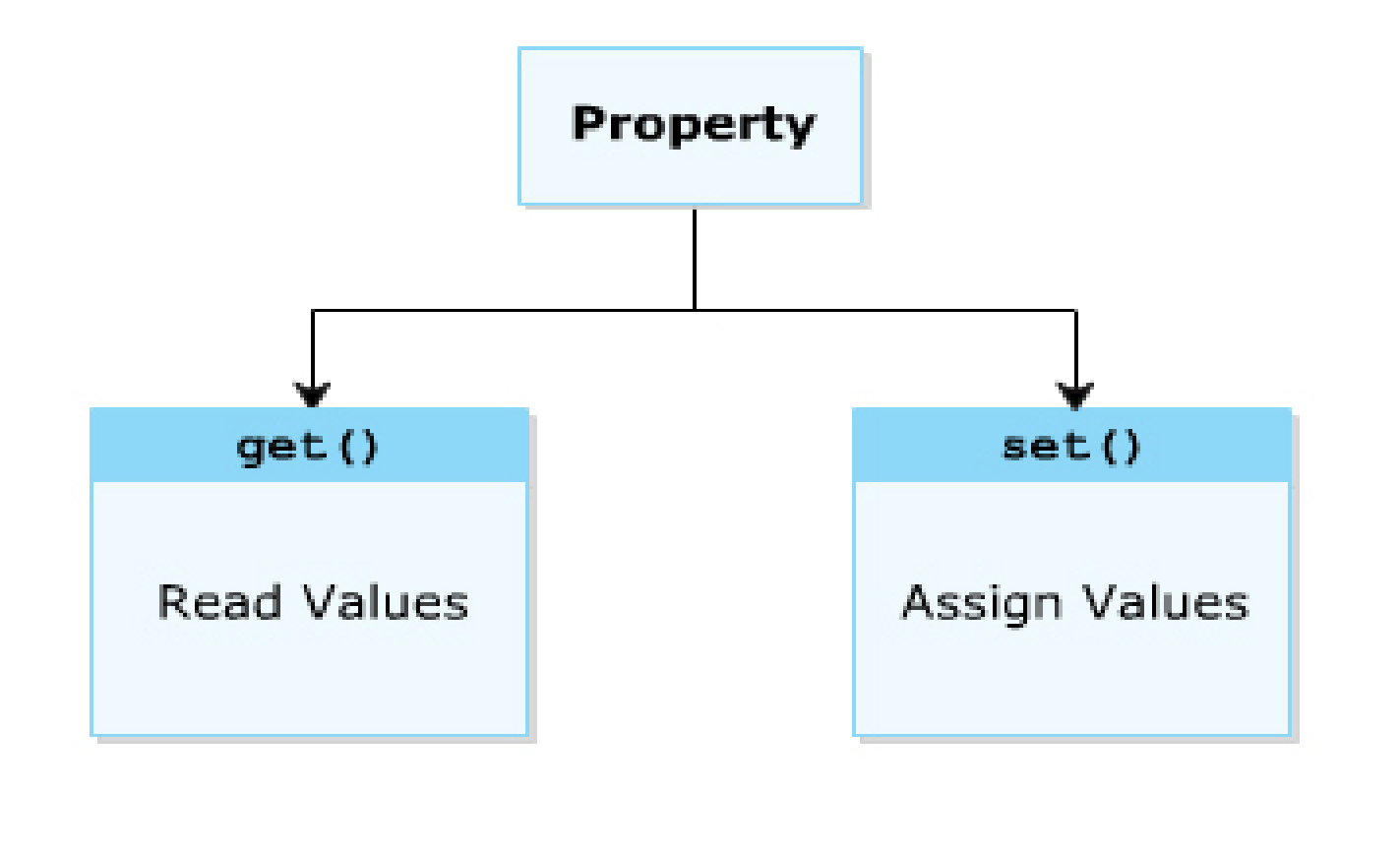
```
class <ClassName>
{
        <ClassName>() //constructor
        {
            //Initialization code
        }
        ~<ClassName>() //destructor
        {
            //De-allocation of objects
        }
}
```

Properties

Properties

- Encapsulate attributes in read/write operators
- Expose a public way of getting and setting values, while hiding implementation or verification code

Properties



```
public class SaleItem{
    private string saleName;
    public string SaleName{
        get{
            return saleName;
        }
        set{
            saleName = value;
        }
    }
    private decimal salePrice;
    public decimal SalePrice{
        get{
            return salePrice;
        }
        set{
            salePrice = value;
        }
    }

    public SaleItem(string saleName, string salePrice){
        this.saleName = saleName;
        this.salePrice = salePrice;
    }
}
```

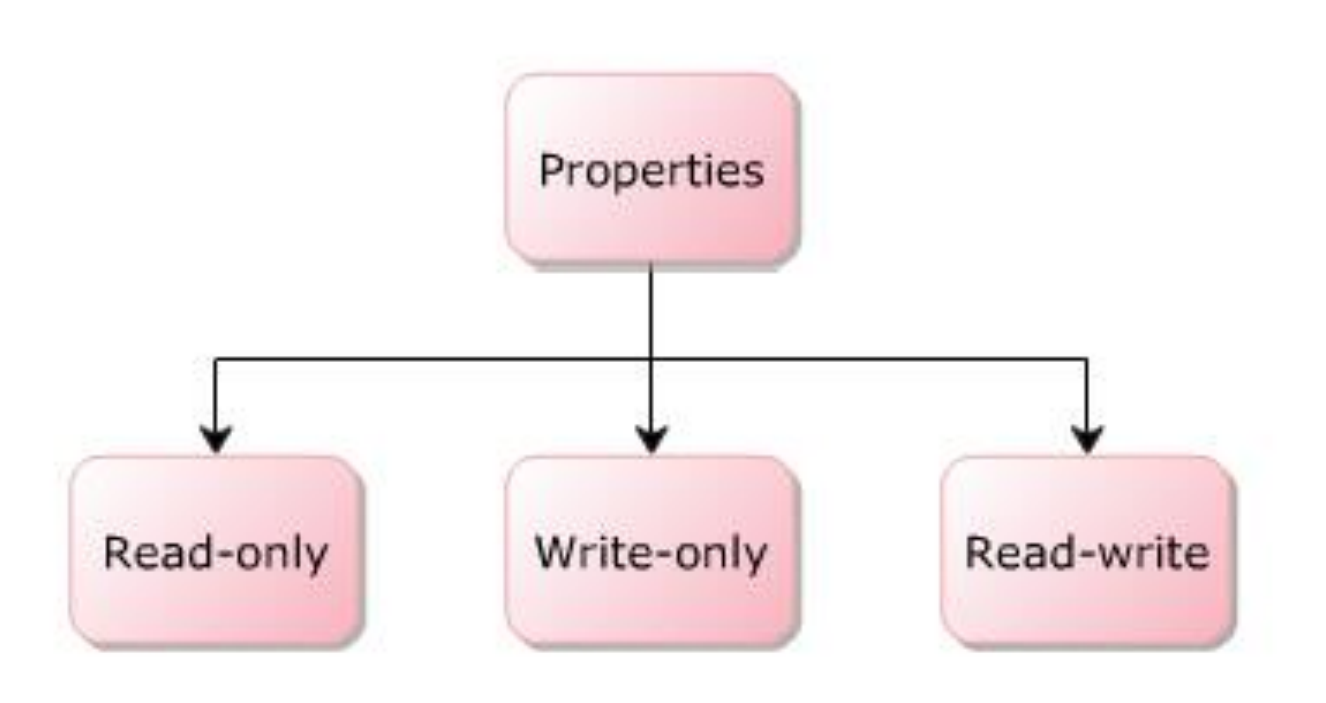
```
public class Person{
    private string firstName;
    private string lastName;

    public string FullName{
        get{
            return firstName + " " + lastName;
        }
    }

    public Person(string firstName, string lastName){
        this.firstName = firstName;
        this.lastName = lastName;
    }

    public override string ToString(){
        return FullName;
    }
}
```

Properties



Content

- Classes
- **Inheritance**
- Interface
- Summary

Inheritance

- Derive a class from another class for a hierarchy of classes that share a set of attributes and methods
- **Example**:
  - Frog derives from aquatic animal
- Syntax

  class <*Sub class name*>**:**<*Base class*>{

  }
- C# does not support multiple inheritance of classes

Inheritance

```
class Animal
{
    public void Eat()
    {
        Console.WriteLine("Every animal eats something");
    }

    public void Do()
    {
        Console.WriteLine("Every animal does something");
    }
}

class Cat:Animal
{
    static void Main()
    {
        Cat obj = new Cat();
        obj.Eat();
        obj.Do();
        Console.ReadLine();
    }
}
```

Method overriding

Method Overriding

- A subclass provides a specific implementation of a method that is already provided by its super class
- The overridden method must be **virtual, abstract, override**
- The **override** modifier is used in the overridden method

Method Overriding

```
class Animal
{
    public virtual void Eat()
    {
        Console.WriteLine("Every animal eats something");
    }

    public void Do()
    {
        Console.WriteLine("Every animal does something");
    }
}

class Cat:Animal
{
    public override void Eat()
    {
        base.Eat();
        Console.WriteLine("Cat eat mice");
    }
    static void Main()
    {
        Cat obj = new Cat();
        obj.Eat();
        obj.Do();
        Console.ReadLine();
    }
}
```

The keyword base

- Used to access members of the base class from within a derived class
- Usages
  - Call a method on the base class that has been overridden by another method
  - Specify which base-class constructor should be called when creating instances of the derived class

Content

- Classes
- Inheritance
- **Interface**
- Summary

Interface

Interface

- Interface enforces certain properties of an object in a context
  - A person
    - At home: a child
    - At school: a student
    - At class: a friend
- Kinda like “standard” to what something should be implemented

Interface

- Declaration

  interface Name {...}
- An interface only contains method declaration
- It can be derived from another interface
- A class can implement one or more interfaces

Interface

```
interface ITerrestrialAnimal
{
    void Eat();
}
interface IMarineAnimal
{
    void Swim();
}

class Crocodile : ITerrestrialAnimal, IMarineAnimal
{
    public void Eat()
    {
      Console.WriteLine("The Crocodile eats flesh");
    }

    public void Swim()
    {
      Console.WriteLine("The Crocodile can swim four
times faster than an Olympic swimmer");
    }
    static void Main(string[] args)
    {
      Crocodile objCrocodile = new Crocodile();
      objCrocodile.Eat();
      objCrocodile.Swim();
    }
}
```

Content

- Classes
- Inheritance
- Interface
- **Summary**

Summary

- C#: an OOP language
- Property: field encapsulation
- Interface: a contract that defines a set of methods a class must implement